

Rust en Sistemas Críticos

Cuando los fallos cuestan dinero... o vidas



Florian Gilcher · Co-fundador, Ferrous Systems & Rust Foundation

Eloy Rubio Suárez, Andrea Ivanov García, Alejandro González Álvarez

Mission Critical vs Safety Critical

MISSION CRITICAL

El fallo tiene impacto económico enorme

- Datacenter caído
- Gestión de Redes Eléctricas
- Servicio financiero interrumpido

SAFETY CRITICAL

El fallo puede costar vidas humanas

- Freno de un coche
- Software de aviación
- Dispositivos médicos



La frontera es cada vez más borrosa: un coche conectado es a la vez safety critical y potencialmente vulnerable a ciberataques

Lo que el mundo web no tiene

IEC 61508

Industria general

ISO 26262

Automoción (ASIL A→D)

DO-178

Aviación

MISRA

Guías de codificación



ISO 26262 recomienda explícitamente usar un lenguaje de tipado estático si está disponible — abriendo la puerta oficial a Rust

Nació en Mozilla. Funciona en aviones.

01 Memory safety en compilación

Sin GC ni runtime. Elimina dangling pointers, buffer overflows y data races por diseño.

02 Sin runtime

Equivalente a C en ejecución. Esencial en sistemas embebidos sin SO debajo.

03 Ownership & Borrowing

Si compila, hay garantía sólida de que la gestión de memoria es correcta.

04 Interop con C

Migración incremental función a función. No hace falta reescribir todo de golpe.

El compilador cualificado para sistemas certificados



Downstream oficial de Rust

Toma cambios del upstream cada noche. No es un fork independiente.



Tests trazados a la spec

Validación en microcontroladores reales: Cortex-M, RISC-V.



Soporte a largo plazo

Por versión. No "actualiza cada 6 semanas o sin soporte".



Primera especificación formal

Aceptada oficialmente por el proyecto Rust. Elimina un bloqueo histórico.

MODELO DE NEGOCIO



Open source

Código fuente y proceso públicos en GitHub



Versión cualificada

Requiere empresa con responsabilidad contractual



Soporte garantizado

Respuesta en plazo — imposible con voluntarios

Antes de nada, pregúntate...

¿Hay razón de peso para hacer un cambio?



¿Tienes los recursos?

→ Buen equipo y tiempo

→ Empieza por algo pequeño

VENTAJAS

- Si no llegas a tiempo, ya tienes algo hecho
- Interoperabilidad
- No supone gran cambio generacional

DESVENTAJAS

- Pérdida de sabiduría técnica
- Curva de aprendizaje muy empinada

CONCLUSIÓN

"Rust se convertirá en el lenguaje estándar para sistemas safety critical"

— Florian Gilcher